

Image Classification using Convolutional Neural Network (CNN)

1. Objective

The objective of this task was to develop a Convolutional Neural Network (CNN) capable of classifying handwritten digit images from the MNIST dataset. The project focused on understanding convolutional layers, pooling layers, dense layers, regularization techniques, and evaluating the model using various performance metrics.

2. Tools and Libraries Used

- Python
- TensorFlow
- Keras
- NumPy
- Matplotlib
- Scikit-learn
- Seaborn
- Google Colab

3. Dataset Description

The MNIST dataset is a benchmark dataset widely used for image classification tasks. It consists of grayscale images of handwritten digits ranging from 0 to 9.

Dataset Details:

- Training Images: 60,000

- Testing Images: 10,000
- Image Size: 28×28 pixels
- Number of Classes: 10

4. Data Preprocessing

The following preprocessing techniques were applied before training the model:

- Loaded the MNIST dataset.
- Normalized pixel values between 0 and 1.
- Reshaped images to include the channel dimension.
- Applied data augmentation techniques including:
 - Rotation
 - Width shifting
 - Height shifting
 - Zooming

These preprocessing steps improved the robustness and generalization capability of the CNN model.

5. CNN Architecture

The CNN model consisted of the following layers:

- Conv2D (32 filters, ReLU activation)
- MaxPooling2D
- Conv2D (64 filters, ReLU activation)
- MaxPooling2D
- Flatten Layer
- Dense Layer (64 neurons, ReLU activation)
- Dropout Layer (0.5)
- Output Layer (10 neurons, Softmax activation)

The Dropout layer was introduced to reduce overfitting by randomly disabling neurons during training.

6. Hyperparameter Configuration

The following hyperparameters were used:

- Optimizer: Adam
- Loss Function: Sparse Categorical Crossentropy
- Epochs: 5
- Batch Size: 32
- Activation Function: ReLU
- Output Activation: Softmax

7. Model Training

The CNN model was trained using the augmented training dataset. During training, both training and validation accuracy and loss were monitored to evaluate the learning process and identify any signs of overfitting.

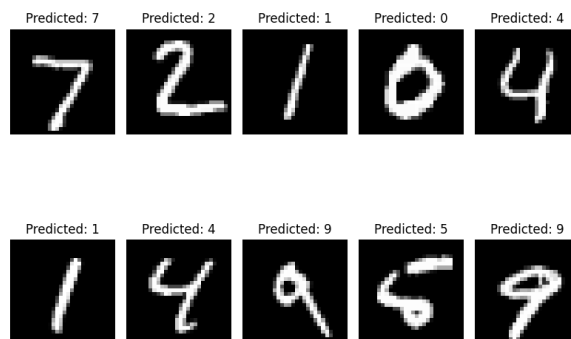
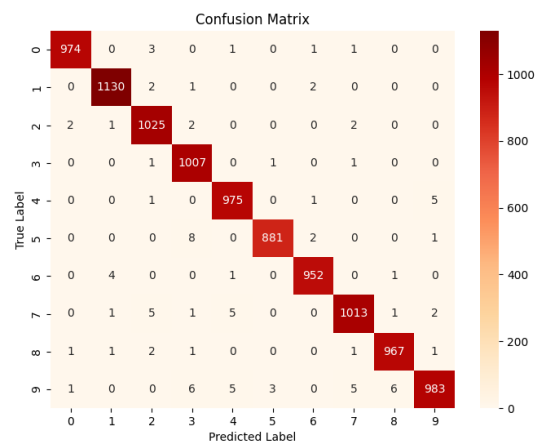
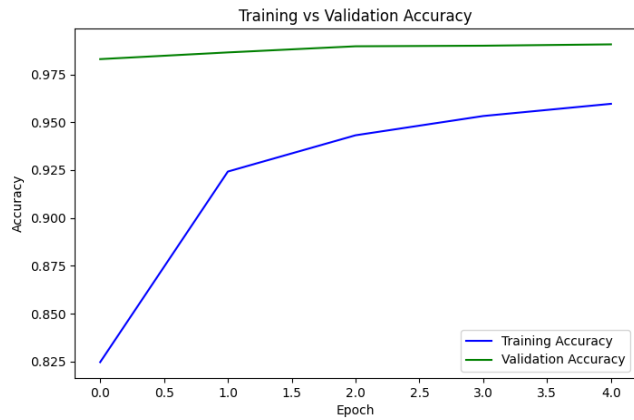
8. Model Evaluation

The trained model was evaluated on the test dataset using multiple evaluation metrics.

Evaluation methods included:

- Test Accuracy
- Test Loss
- Confusion Matrix
- Classification Report

- Accuracy Curve
- Loss Curve



Model: "sequential_1"		
Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_2 (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_3 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_3 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten_1 (Flatten)	(None, 1600)	0
dense_2 (Dense)	(None, 64)	102,464
dropout_1 (Dropout)	(None, 64)	0
dense_3 (Dense)	(None, 10)	650
Total params: 121,930 (476.29 KB)		
Trainable params: 121,930 (476.29 KB)		
Non-trainable params: 0 (0.00 B)		

CNN model summary

	precision	recall	f1-score	support
0	1.00	0.99	0.99	980
1	0.99	1.00	0.99	1135
2	0.99	0.99	0.99	1032
3	0.98	1.00	0.99	1010
4	0.99	0.99	0.99	982
5	1.00	0.99	0.99	892
6	0.99	0.99	0.99	958
7	0.99	0.99	0.99	1028
8	0.99	0.99	0.99	974
9	0.99	0.97	0.98	1009
accuracy			0.99	10000
macro avg	0.99	0.99	0.99	10000
weighted avg	0.99	0.99	0.99	10000

Classification Report

Final Performance

- **Test Accuracy:** 99.07%
- **Test Loss:** 0.0269%

The confusion matrix indicated that the majority of handwritten digits were classified correctly, while the classification report showed high precision, recall, and F1-score across all classes.

9. Results and Discussion

The CNN achieved high accuracy on the MNIST dataset. The training and validation accuracy increased steadily during training, while both loss values decreased consistently. The inclusion of data augmentation and dropout improved the model's ability to generalize to unseen data.

The confusion matrix showed very few misclassifications, and the classification report confirmed strong performance across all digit classes.

10. Conclusion

This project successfully demonstrated the implementation of a Convolutional Neural Network for handwritten digit classification using the MNIST dataset. The use of convolutional layers enabled effective feature extraction, while data augmentation and dropout improved generalization performance. The model achieved high classification accuracy and provided practical experience in designing, training, evaluating, and saving deep learning models.